

Google Colab link to assignment

[https://colab.research.google.com/
drive/1RngjwNmeTDqUOTbEdgQP
0tFj7J-4h67v](https://colab.research.google.com/drive/1RngjwNmeTDqUOTbEdgQP0tFj7J-4h67v)

Screenshots attached below

Assignment No. 2: Data Analysis using NumPy and Pandas

Name: Nandini Shirude

Class: IT2, I21

Subject: Essentials of Data Science

Objective

To create and solve 20 problem statements using NumPy and Pandas on a given dataset, and to analyze the data for meaningful insights.

Dataset Description

The dataset used in this analysis contains information about sales transactions, including details such as sales amount, quantity sold, product category, region, discount, and payment method. It is used to analyze sales performance and identify useful insights

Section A: Data Loading and Exploration

```
import pandas as pd
import numpy as np
df = pd.read_csv('sales_data.csv')
df.head()
```

Product_ID	Sale_Date	Sales_Rep	Region	Sales_Amount	Quantity_Sold	Product_Category	Unit_Cost	Unit_Price	Customer_Type	Discount	Payment_Method	Sales_Channel	Region_and_Sales_Rep
1052	2023-02-03	Bob	North	5053.97	18	Furniture	152.75	267.22	Returning	0.09	Cash	Online	North-Bob
1093	2023-04-21	Bob	West	4384.02	17	Furniture	3816.39	4209.44	Returning	0.11	Cash	Retail	West-Bob
1015	2023-09-21	David	South	4631.23	30	Food	261.56	371.40	Returning	0.20	Bank Transfer	Retail	South-David
1072	2023-08-24	Bob	South	2167.94	39	Clothing	4330.03	4467.75	New	0.02	Credit Card	Retail	South-Bob
1061	2023-03-24	Charlie	East	3750.20	13	Electronics	637.37	692.71	New	0.08	Credit Card	Online	East-Charlie

```
[ ] #Q1- Display the first five records of the dataset to understand its structure and contents.  
df.head()
```

...	Product_ID	Sale_Date	Sales_Rep	Region	Sales_Amount	Quantity_Sold	Product_Category	Unit_Cost	Unit_Price	Customer_Type	Discount	Payment_Method	Sales_Channel	Region_and_Sales_Rep
	1052	2023-02-03	Bob	North	5053.97	18	Furniture	152.75	267.22	Returning	0.09	Cash	Online	North-Bob
	1093	2023-04-21	Bob	West	4384.02	17	Furniture	3816.39	4209.44	Returning	0.11	Cash	Retail	West-Bob
	1015	2023-09-21	David	South	4631.23	30	Food	261.56	371.40	Returning	0.20	Bank Transfer	Retail	South-David
	1072	2023-08-24	Bob	South	2167.94	39	Clothing	4330.03	4467.75	New	0.02	Credit Card	Retail	South-Bob
	1061	2023-03-24	Charlie	East	3750.20	13	Electronics	637.37	692.71	New	0.08	Credit Card	Online	East-Charlie

```
[ ] #Q2- Display the last five records of the dataset to examine how the data is organized at the end.  
df.tail()
```

...	Product_ID	Sale_Date	Sales_Rep	Region	Sales_Amount	Quantity_Sold	Product_Category	Unit_Cost	Unit_Price	Customer_Type	Discount	Payment_Method	Sales_Channel	Region_and_Sales
995	1010	2023-04-15	Charlie	North	4733.88	4	Food	4943.03	5442.15	Returning	0.29	Cash	Online	North-C
996	1067	2023-09-07	Bob	North	4716.36	37	Clothing	1754.32	1856.40	New	0.21	Bank Transfer	Retail	North
997	1018	2023-04-27	David	South	7629.70	17	Clothing	355.72	438.27	Returning	0.06	Bank Transfer	Online	South-I
998	1100	2023-12-20	David	West	1629.47	39	Electronics	3685.03	3743.39	New	0.01	Bank Transfer	Online	West-I
999	1086	2023-08-16	Alice	East	4923.93	48	Food	2632.58	2926.68	Returning	0.14	Cash	Online	East

```
#Q3- Determine the total number of rows and columns present in the dataset.
df.shape
```

```
(1000, 14)
```

```
#Q4- Retrieve and display the names of all the columns in the dataset.
df.columns
```

```
Index(['Product_ID', 'Sale_Date', 'Sales_Rep', 'Region', 'Sales_Amount',
       'Quantity_Sold', 'Product_Category', 'Unit_Cost', 'Unit_Price',
       'Customer_Type', 'Discount', 'Payment_Method', 'Sales_Channel',
       'Region_and_Sales_Rep'],
      dtype='object')
```

Section B: Data Understanding

```
#Q5- Obtain a concise summary of the dataset including data types and non-null values.
df.info()
```

```
*** <class 'pandas.core.frame.DataFrame'>
RangeIndex: 1000 entries, 0 to 999
Data columns (total 14 columns):
 #   Column                Non-Null Count  Dtype  
---  -
 0   Product_ID            1000 non-null  int64  
 1   Sale_Date             1000 non-null  object  
 2   Sales_Rep             1000 non-null  object  
 3   Region                1000 non-null  object  
 4   Sales_Amount          1000 non-null  float64 
 5   Quantity_Sold         1000 non-null  int64  
 6   Product_Category      1000 non-null  object  
 7   Unit_Cost             1000 non-null  float64 
 8   Unit_Price            1000 non-null  float64 
 9   Customer_Type         1000 non-null  object  
10   Discount              1000 non-null  float64 
11   Payment_Method        1000 non-null  object  
12   Sales_Channel         1000 non-null  object  
13   Region_and_Sales_Rep  1000 non-null  object  
dtypes: float64(4), int64(2), object(8)
memory usage: 109.5+ KB
```

```
#Q6- Identify the total number of missing values in each column and determine which column has the highest number of missing entries.
missing = df.isnull().sum()
missing
missing.idxmax()
```

'Product_ID'

```
#Q7- Determine the percentage of missing values in each column to better understand data quality.
(df.isnull().sum() / len(df)) * 100
```

```
...
0
Product_ID    0.0
Sale_Date     0.0
Sales_Rep     0.0
Region        0.0
Sales_Amount  0.0
Quantity_Sold 0.0
Product_Category 0.0
Unit_Cost     0.0
Unit_Price    0.0
Customer_Type 0.0
Discount      0.0
```

Region_and_Sales_Rep 0.0

dtype: float64

```
#Q8- Analyze the distribution of numerical features by generating descriptive statistics and interpreting variability in the data.
df.describe()
```

```
...
Product_ID Sales_Amount Quantity_Sold Unit_Cost Unit_Price Discount
count 1000.000000 1000.000000 1000.000000 1000.000000 1000.000000 1000.000000
mean 1050.128000 5019.265230 25.355000 2475.304550 2728.440120 0.15239
std 29.573505 2846.790126 14.159006 1417.872546 1419.399839 0.08720
min 1001.000000 100.120000 1.000000 60.280000 167.120000 0.00000
25% 1024.000000 2550.297500 13.000000 1238.380000 1509.085000 0.08000
50% 1051.000000 5019.300000 25.000000 2467.235000 2696.400000 0.15000
75% 1075.000000 7507.445000 38.000000 3702.865000 3957.970000 0.23000
max 1100.000000 9989.040000 49.000000 4995.300000 5442.150000 0.30000
```


Section C: NumPy Operations

```
#Q9- Convert the 'Sales_Amount' column into a NumPy array and compute its mean, median, and standard deviation.
sales = np.array(df['Sales_Amount'])
np.mean(sales), np.median(sales), np.std(sales)
```

```
(np.float64(5019.265229999999),  
 np.float64(5019.3),  
 np.float64(2845.3663745785966))
```

```
sales = np.array(df['Sales_Amount'])
np.max(sales), np.min(sales)
```

```
(np.float64(9989.04), np.float64(100.12))
```

▶ #Q11- Identify all sales values that are greater than the average sales amount using NumPy filtering.

```
sales[sales > np.mean(sales)]
```

```
... array([5053.97, 7698.92, 8239.58, 8518.45, 6607.8 , 8813.55, 6810.35,
        6116.75, 6551.23, 7412.11, 6483.84, 7160.75, 8913.13, 5207.03,
        8371.25, 5356.28, 5991.8 , 9638.64, 5238.42, 6807.67, 7762.51,
        7751.92, 5260.83, 9762.54, 7724.57, 8090.84, 7754.1 , 5227.81,
        6499.94, 9744.52, 8485.9 , 8995.75, 7853.66, 7825.72, 7499.7 ,
        9582.12, 9432.97, 7873.38, 9914.15, 5490.38, 9631.41, 6261.9 ,
        7835.09, 5825.15, 8130.13, 6395.95, 6773.89, 9183.11, 8912.4 ,
        7106.44, 6346.13, 9583.55, 5343.63, 5055.98, 6828.24, 8088.96,
        5511.11, 8452.13, 9813.81, 7952.11, 8552.47, 9610.21, 9519.16,
        9948.71, 7146.06, 9813.32, 5738.44, 5976.25, 6261.2 , 9813.66,
        7946.69, 9088.2 , 9442.65, 9605.34, 5262.45, 9775.35, 7597.37,
        7211.48, 6442.09, 6699.34, 5694.34, 7739.11, 5039.07, 9269.32,
        6380.2 , 8495.62, 7315.73, 6658.1 , 9203.36, 9933.22, 7633.98,
        7567.22, 9192.42, 9514.19, 5813.51, 7896.74, 5684.33, 6600.65,
        9190.57, 5108.98, 8681.03, 9509.55, 5858.06, 5842.88, 5215.31,
        7611.88, 8951.53, 9469.92, 8936.33, 7825.62, 5025.64, 5952.19,
        8564.8 , 9413.77, 5118.51, 8127.7 , 8374.68, 6966.82, 7734.12,
        6581.04, 8771.24, 5437.04, 7678.91, 5272.85, 9733.47, 7320.51,
        9671.77, 6664.17, 7444.77, 8499.41, 7154.95, 8186.12, 9029.21,
        8086.27, 9850.1 , 7567.06, 5947.31, 6644.04, 5490.52, 7122.28,
        7828.25, 5879.35, 9527. , 6055.38, 7172.8 , 7614.15, 5187.16,
        5138.02, 9335.96, 8292.31, 7002.37, 7171.83, 9217.85, 6976.49,
        7316.91, 8630.74, 8090. , 9787.4 , 8579.72, 7041.58, 7297.86,
        5664.52, 9476.2 , 7883.44, 7370.99, 8406.07, 7066.63, 9705.55,
        7078.22, 8189.57, 5509.66, 9673.65, 5097.48, 7212.69, 8640.14,
        9888.82, 5574.8 , 8665.43, 8733. , 6585.36, 7823.45, 6886.25,
```

Section D: Pandas Analysis

#Q12- Calculate the total sales amount for each region to understand regional performance.
`df.groupby('Region')['Sales_Amount'].sum()`

```
...
      Sales_Amount
Region
East    1259792.93
North   1369612.51
South   1154250.86
West    1235608.93
```

`dtype: float64`

#Q13- Find the region with the highest total sales amount.
`df.groupby('Region')['Sales_Amount'].sum().idxmax()`

`'North'`

#Q14- Calculate the average sales amount for each product category.
`df.groupby('Product_Category')['Sales_Amount'].mean()`

```
...
      Sales_Amount
Product_Category
Clothing    4901.023731
Electronics 5054.876585
Food        5317.582035
Furniture   4848.144962
```

`dtype: float64`

Section E: Grouping and Aggregation

#Q15- Find the product category with the highest average sales amount.
`df.groupby('Product_Category')['Sales_Amount'].mean().idxmax()`

`'Food'`

Product_Category	Quantity_Sold
Clothing	6922
Electronics	6096
Food	5608
Furniture	6729

	Product_ID	Sale_Date	Sales_Rep	Region	Sales_Amount	Quantity_Sold	Product_Category	Unit_Cost	Unit_Price	Customer_Type	Discount	Payment_Method	Sales_Channel	Region_and_Sale
0	1052	2023-02-03	Bob	North	5053.97	18	Furniture	152.75	267.22	Returning	0.09	Cash	Online	North
7	1087	2023-01-06	Eve	South	7698.92	46	Furniture	3702.51	3964.65	New	0.12	Bank Transfer	Online	South

[illegible]

#Q18- Filter the dataset to show transactions where the discount is greater than 15%.
`df[df['Discount'] > 0.15]`

	Product_ID	Sale_Date	Sales_Rep	Region	Sales_Amount	Quantity_Sold	Product_Category	Unit_Cost	Unit_Price	Customer_Type	Discount	Payment_Method	Sales_Channel	Region_and_Sale
2	1015	2023-09-21	David	South	4631.23	30	Food	261.56	371.40	Returning	0.20	Bank Transfer	Retail	South
5	1021	2023-02-11	Charlie	West	3761.15	32	Food	900.79	1106.51	New	0.21	Cash	Online	West-C
10	1088	2023-11-16	Eve	North	8518.45	13	Furniture	2440.11	2517.60	New	0.23	Bank Transfer	Retail	Nor
13	1003	2023-12-31	Alice	South	4775.59	30	Furniture	4190.28	4270.65	New	0.20	Cash	Online	South
14	1022	2023-08-17	Charlie	South	8813.55	21	Food	2537.20	2869.60	New	0.29	Bank Transfer	Retail	South-C
...	...	...	...	...	...	...	...	...	...	...	...	...	...	...
992	1084	2023-02-19	David	West	2154.66	35	Food	465.61	812.91	New	0.16	Bank Transfer	Online	West
993	1025	2023-06-26	David	North	2457.65	47	Furniture	3861.61	3998.91	New	0.25	Credit Card	Online	North
994	1068	2023-04-06	Alice	South	9093.50	31	Clothing	3169.37	3304.15	Returning	0.25	Credit Card	Retail	South
995	1010	2023-04-15	Charlie	North	4733.88	4	Food	4943.03	5442.15	Returning	0.29	Cash	Online	North-C
996	1067	2023-09-07	Bob	North	4716.36	37	Clothing	1754.32	1856.40	New	0.21	Bank Transfer	Retail	North

17	...	...	...	...	...	...	...	...	...	...	...	...	...	...
...	...	...	...	...	...	...	...	...	...	...	...	...	...	...
992	1084	2023-02-19	David	West	2154.66	35	Food	465.61	812.91	New	0.16	Bank Transfer	Online	West-D
993	1025	2023-06-26	David	North	2457.65	47	Furniture	3861.61	3998.91	New	0.25	Credit Card	Online	North-D
994	1068	2023-04-06	Alice	South	9093.50	31	Clothing	3169.37	3304.15	Returning	0.25	Credit Card	Retail	South-A
995	1010	2023-04-15	Charlie	North	4733.88	4	Food	4943.03	5442.15	Returning	0.29	Cash	Online	North-Ch
996	1067	2023-09-07	Bob	North	4716.36	37	Clothing	1754.32	1856.40	New	0.21	Bank Transfer	Retail	North-

18 rows x 14 columns

Section G: Feature Engineering

#Q19- Create a new column named 'Profit' and determine the average profit generated for each region.

```
df['Profit'] = (df['Unit_Price'] - df['Unit_Cost']) * df['Quantity_Sold']
df.groupby('Region')['Profit'].mean()
```

Profit
Region

```
#Q19- Create a new column named 'Profit' and determine the average profit generated for each region.
```

```
df['Profit'] = (df['Unit_Price'] - df['Unit_Cost']) * df['Quantity_Sold']  
df.groupby('Region')['Profit'].mean()
```

	Profit
Region	
East	6275.882890
North	6222.701124
South	6724.499558
West	6787.261352

dtype: float64

```
#Q20- Calculate the total profit generated from all transactions.  
df['Profit'].sum()
```

```
... np.float64(6487847.07)
```

Conclusion

In this project, the dataset was analyzed using NumPy and Pandas to understand sales patterns and performance. Key operations like grouping, filtering, and calculations helped in finding useful insights such as total sales, profit, and regional trends. This shows how data analysis tools can be used to make better decisions.